

# Métodos de Aproximación

*Calculadora Científica Graficadora*

Pensamiento Algorítmico

## Introducción

El código de la Calculadora Científica Graficadora no usa ninguna librería externa de Python. Todas las funciones matemáticas están construidas desde cero usando cuatro métodos de aproximación numérica. A continuación se explica en qué consiste cada uno y en qué parte del código se usa.

## 1. Series de Taylor

La idea central de este método es que casi cualquier función matemática puede escribirse como una suma de términos que siguen un patrón. Cada término es una potencia de  $x$  dividida entre el factorial de un número. Mientras más términos se sumen, más exacto es el resultado.

Por ejemplo, el seno de  $x$  se calcula así:

$$\sin(x) = x - x^3/3! + x^5/5! - x^7/7! + \dots$$

El código suma 15 términos de esa serie, lo cual es suficiente para obtener seis decimales de precisión. El mismo enfoque se aplica para calcular el coseno, la exponencial ( $e^x$ ), el arcoseno y el arcotangente.

### Se usa en las funciones:

- `exponencial(x)`: calcula  $e^x$  sumando la serie hasta el término 80.
- `seno(x)` y `coseno(x)`: suman 15 términos alternando signos positivos y negativos.
- `arcoseno(x)`: usa una variante de la serie que funciona cuando  $x$  está entre  $-1$  y  $1$ .
- `arcotangente(x)`: suma 100 términos de su propia serie.

## 2. Método de Newton-Raphson

Este método sirve para encontrar el valor que resuelve una ecuación, cuando no hay una fórmula directa para calcularlo. La estrategia es sencilla: se parte de un valor estimado y se aplica una fórmula que lo mejora poco a poco, repitiendo el proceso hasta que el resultado ya no cambie.

En el caso de la raíz n-ésima, el problema es encontrar un número  $r$  tal que  $r^n = x$ . La fórmula de mejora que usa el código es:

$$r\_nuevo = ((n - 1) \cdot r + x / r^{n-1}) / n$$

El código repite este cálculo 200 veces, lo que garantiza que el resultado converja con alta precisión.

**Se usa en las funciones:**

- `raiz_n(x, n)`: calcula la raíz n-ésima con 200 iteraciones del método de Newton.
- `raiz_cuadrada(x)`: es un caso particular del mismo método (conocido como el método babilónico), con 100 iteraciones. Converge tan rápido que pocas iteraciones ya son suficientes.

### 3. Serie del arctanh para el logaritmo natural

Calcular el logaritmo natural con una serie de Taylor directa funciona, pero converge muy lentamente, es decir, necesitaría miles de términos para ser preciso. El código usa un truco matemático: transforma el problema usando la identidad:

$$\ln(x) = 2 \cdot \operatorname{arctanh}((x - 1) / (x + 1))$$

El `arctanh` tiene una serie que converge mucho más rápido. Además, antes de aplicarla, el código reduce  $x$  a un rango cercano a 1 (entre 0.5 y 2) multiplicándolo o dividiéndolo por  $e$  las veces que haga falta. Esto garantiza que la serie converja bien con solo 500 términos.

**Se usa en las funciones:**

- `logaritmo_natural(x)`: aplica la reducción de rango y luego suma 500 términos de la serie del `arctanh`.
- `logaritmo_base(x, base)`: llama a `logaritmo_natural` dos veces y divide los resultados.
- `log10(x)`: caso particular de `logaritmo_base` con base 10.

### 4. Reducción de ángulo

Este no es un método de aproximación en sí, sino una estrategia que mejora la precisión de los demás métodos. Las series de Taylor para seno y coseno funcionan bien cuando  $x$  es un número pequeño, cercano a cero. Si alguien ingresa un ángulo grande, como 1000 radianes, los errores se acumulan y el resultado puede ser incorrecto.

Para evitarlo, el código aprovecha que el seno y el coseno se repiten cada  $2\pi$  (aproximadamente 6.28). Esto significa que  $\sin(1000)$  es igual a  $\sin(1000 - 159 \times 2\pi)$ , que es un número pequeño. Antes de calcular, el código siempre resta los múltiplos completos de  $2\pi$ , dejando solo el residuo.

**Se usa en las funciones:**

- $\text{seno}(x)$ : reduce  $x$  al rango  $[-2\pi, 2\pi]$  antes de aplicar la serie de Taylor.
- $\text{coseno}(x)$ : aplica la misma reducción.